7.1. Data Merging & Joining

Previous Section:

6.2. Data Encoding

Contents:

In many real-world applications, information is distributed across multiple tables rather than being stored in a single dataset. For example, a university may store student information, instructor information, and course information in separate tables. To perform meaningful analysis, these datasets often need to be combined based on one or more common columns.

In this section, we create three datasets:

- **Student** – Contains student information such as age, gender, major, and GPA.
- **Instructor** – Contains instructor details.
- **Course** – Contains course information, including the instructor responsible for teaching each course.

Student Dataset

```
import numpy as np
import pandas as pd

np.random.seed(42)

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

majors = ["AI", "IT", "CSC", "SW"]
gender = ["Male", "Female"]

n = 20

students = pd.DataFrame({
    "StudentID": range(1001, 1001+n),
    "Name": names,
    "Gender": np.random.choice(gender, n),
    "Age": np.random.randint(18, 22, n),
```

```
    "Major": np.random.choice(majors, n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

print(students.head())
```

Instructor Dataset

```
instructors = pd.DataFrame({
    "InstructorID": [101,102,103,104,105],
    "Instructor": [
        "Dr. Smith",
        "Dr. Johnson",
        "Dr. Lee",
        "Dr. Brown",
        "Dr. Davis"
    ],
    "Age": [45, 52, 38, 41, 50],
    "Gender": ["Male","Female","Female","Male","Male"]
})

print(instructors)
```

Course Dataset

```
courses = pd.DataFrame({
    "CourseID": [1,2,3,4,5,6],
    "Course": [
        "Machine Learning",
        "Basic Programming",
        "Data Analytics",
        "Software Engineering",
        "Computer Vision",
        "Web Development"
    ],
    "Major": ["AI","IT","CSC","SW","AI","IT"],
```

```
    "Semester": [1,1,2,2,2,1],
    "Time": [
        "Mon 08:00",
        "Tue 10:00",
        "Wed 13:00",
        "Thu 09:00",
        "Fri 14:00",
        "Fri 08:00"
    ],
    "InstructorID": [101,102,103,104,105,102]
})

print(courses)
```

1. Data Merging

For merging, Pandas provides the `merge()` function, which behaves similarly to SQL JOIN operations. It allows two DataFrames to be combined using one or more common columns while supporting different types of joins such as **inner**, **left**, **right**, and **outer** joins.

The `merge()` method is one of the most frequently used functions in data analytics because relational datasets are often stored in separate tables.

Mastering `merge()` makes it much easier to integrate information from multiple data sources.

Note: Since each major can have multiple courses, a student may match multiple rows in the **Course** dataset. During the merge, Pandas creates one row for every valid match, so the resulting DataFrame may contain **more rows than the original student dataset**. This ensures that all matching records are preserved.

1.1. Inner Merge

An **inner merge** returns only the rows that have matching values in both DataFrames.

```
result = students.merge(courses, on="Major", how="inner")
print(result.head())
```

1.2. Left Merge

A **left merge** keeps every row from the left DataFrame. If no matching row exists in the right DataFrame, the missing values are filled with `NaN`.

```
result = students.merge(courses, on="Major", how="left")
print(result.head())
```

1.3. Right Merge

A **right merge** keeps every row from the right DataFrame.

```
result = students.merge(courses, on="Major", how="right")
print(result.head())
```

1.4. Outer Merge

An **outer merge** keeps every row from both DataFrames. Rows without matches are filled with `NaN`.

```
result = students.merge(courses, on="Major", how="outer")
print(result.head())
```

1.5. Merging on Multiple Columns

Sometimes one column is not sufficient to uniquely identify matching records. In this case, multiple columns can be used.

```
student2 = students.copy()

student2["Advisor"] = np.random.choice(
    ["Dr. Smith", "Dr. Johnson"],
    len(student2)
)

advisor = student2[["Major", "Advisor"]].drop_duplicates()

result = student2.merge(
    advisor,
    on=["Major", "Advisor"],
    how="inner"
)

print(result.head())
```

drop_duplicates() is used to ensure duplicate entries are removed

1.6. Merging with Different Column Names

The matching columns do not need to have the same name. Use the `left_on` and `right_on` parameters.

```
courses2 = courses.rename(columns={
    "InstructorID": "ID"
})

result = instructors.merge(
    courses2,
    left_on="InstructorID",
    right_on="ID",
    how="inner"
```

```
)

print(result.head())
```

1.7. Merging Three DataFrames

Multiple DataFrames can be merged sequentially by nesting `merge()` operations.

```
result = (students
        .merge(courses, on="Major", how="left")
        .merge(instructors, on="InstructorID", how="left")
)

print(result.head(10))
```

The resulting DataFrame contains student information together with the courses available for their major and the instructor responsible for teaching each course, demonstrating how multiple datasets can be combined into a single table for analysis.

2. Data Joining

Joining is closely related to merging, and both are used to combine multiple DataFrames into a single dataset. The primary difference is **how the matching is performed**.

- `merge()` matches rows using one or more columns.
- `join()` matches rows using the **index** of each DataFrame.

Because of this, it is common practice to set a meaningful index before performing a join. In most applications, this index is an **ID** column, such as a student ID, employee ID, or product ID. Once the indexes are aligned, `join()` provides a simple and efficient way to combine related datasets.

2.1. Setting the Index

Before joining, set the ID columns as the index.

```
student_idx = students.set_index("StudentID")
course_idx = courses.set_index("InstructorID")
instructor_idx = instructors.set_index("InstructorID")

print(student_idx.head())
print(instructor_idx.head())
```

2.2. Basic Join

The simplest usage joins two DataFrames using their indexes.

```
result = course_idx.join(instructor_idx)
print(result.head())
```

Since both DataFrames use **InstructorID** as the index, Pandas automatically matches the corresponding rows.

2.3. Left Join

By default, `join()` performs a **left join**, preserving all rows from the calling DataFrame.

```
result = course_idx.join(
        instructor_idx,
    how="left"
)

print(result.head())
```

2.4. Inner Join

Only rows with matching indexes are returned.

```
result = course_idx.join(
    instructor_idx,
    how="inner"
)

print(result.head())
```

2.5. Right Join

Keep all rows from the second DataFrame.

```
result = course_idx.join(
    instructor_idx,
    how="right"
)

print(result.head())
```

2.6. Outer Join

Keep every index from both DataFrames.

```
result = course_idx.join(
    instructor_idx,
    how="outer"
)

print(result.head())
```

2.7. Joining Multiple DataFrames

Like `merge()`, `join()` can combine multiple DataFrames in a single operation.

```
result = (course_idx
          .join(instructor_idx, rsuffix="_Instructor")
          .join(student_idx, rsuffix="_Student"))

print(result.head())
```

Note: Although joining multiple DataFrames in this way is possible, it often introduces **missing (`NaN`) values** because the indexes may not perfectly align across all datasets.

In practice, `merge()` is usually the better choice for combining related tables, as it allows you to explicitly specify the matching columns and generally provides more predictable results.

Alternatively, multiple DataFrames can be passed as a list when they share the same index.

```
df1 = student_idx[["Name", "Major"]]
df2 = student_idx[["Age", "GPA"]]

result = df1.join(df2)

print(result.head())
```

Note: When joining DataFrames with overlapping column names, use the `lsuffix` and `rsuffix` parameters to distinguish the duplicate columns.

Summary

Both `merge()` and `join()` are fundamental tools for combining datasets in Pandas.

- Use `merge()` when the matching keys are stored as **columns**.
- Use `join()` when the matching keys are stored as the **index**.
- Both support **left**, **right**, **inner**, and **outer** joins, making them suitable for a wide variety of relational data analysis tasks.

In the next section, we will explore additional methods for combining and organizing data, including **concatenation**, **comparison**, and **segmentation** of DataFrames.

References

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.merge.html

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.join.html

https://www.geeksforgeeks.org/pandas/pandas-merge-dataframe/

https://www.geeksforgeeks.org/python/different-types-of-joins-in-pandas/

Next Section:

7.2. Data Concatenation, Comparison, and Segmentation